

Cognition and Machine Learning

Course Project – Part II

Turgay Yıldız , Cognitive Sciences, Middle East Technical University

Aim of the Project :

The main aim of this project is to understand the role of such concepts as *compositionality*, *causality* and *learning to learn* in Artificial General Intelligence (AGI) by comparing two Convolutional Neural Networks (CNNs) model. Architecture of one of these models was designed so as to employ these concepts, and data was also preprocessed according to this model's architecture. The other model was designed so as to utilize just one-shot classification, without taking into account these aforementioned concepts, and data was also preprocessed according to its architecture.

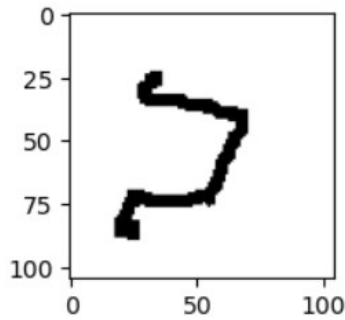
The reason for choosing a Convolutional Neural Network for the project is mainly because these concepts are originally used in application of Bayesian Programming Learning (BPL) which is a generative model or other generative models. The papers I want to compare my results compares how employing these concepts surpasses conventional deep learning models like Convolutional Neural Networks and even human beings in one-shot classification tasks on Omniglot dataset. However, I wanted to see whether or not these concepts are applicable to the Convolutional Neural Networks. Therefore, I designed two different model two compare results both with themselves and with the results from original papers, and also with the baseline model.

First Convolutional Neural Network :

This model was created as a U-NET like network. It composed of two parts: encoder and decoder. This was thought to be the best representation for the data because both input and output have to have the same size (105 x 105 pixels). Therefore, it was chosen as a suitable representation for parsing images of handwritten characters of different alphabets into their strokes. Therefore, the model gets an image with one channel as input and propagates it to 5 channels and yields an image with 5 channels (actually we can say that 5 different images or an array with 5 channels). Outputs are expected to be different strokes of the image. This is like parsing an image into its strokes, like Bayesian Programming Learning does. The results will show whether or not a CNN is able to capture such concepts *compositionality*, *causality* and *learning to learn*.

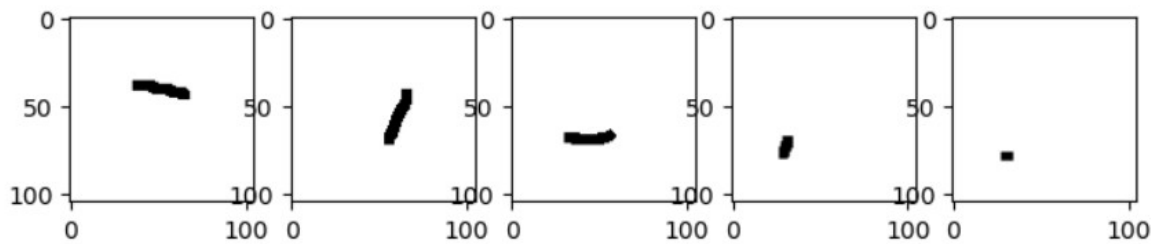
This model takes an image like that:

```
[29]: plot_img(X[189][0], 1, 1, 2, 2)
```



and is expected to return strokes of this image similar to these below :

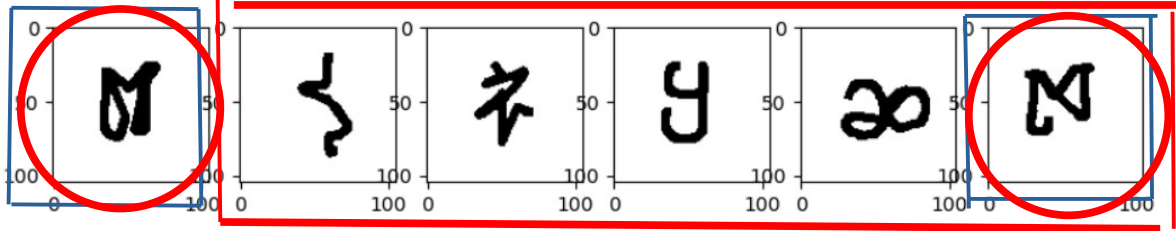
```
[30]: plot_img(y[189], 1, 5, 8, 6)
```



Second Convolutional Neural Network :

This model was also created in two parts. First part is like conventional CNNs. Other part was created so as to have fully connected layers. This was also thought to be the best representation for the data and the aim of the project, because this model takes six different images as input and yields a vector of length of five. Vectors are one-hot vectors. Those arrays of six images are laid out like :

```
[13]: plot_img(X[19],1,6, 10,8 )
```

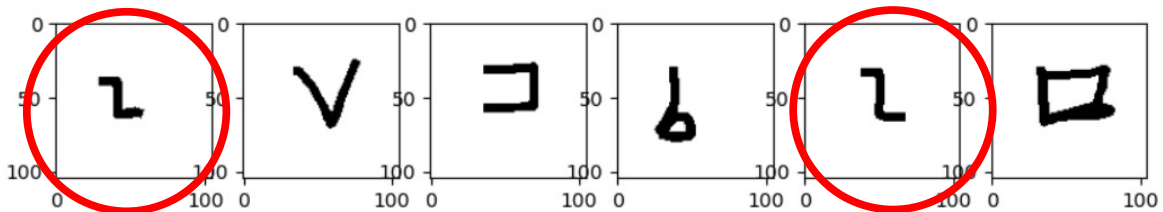


```
[14]: y[19][0]
```

```
[14]: memmap([[0, 0, 0, 0, 1]], dtype=int32)
```

As you can see, first image is the target image, and a similar version (they are not exactly the same) of it randomly inserted into the other 4 images. Here, it is the fifth image ([0, 0, 0, 0, 1]). Moreover, the other four are also randomly chosen so as not to include a similar of the target. Otherwise, one-hot encoding would be meaningless. You can see another example below :

```
[15]: plot_img(X[114],1,6, 10,8 )
```



```
[16]: y[114][0]
```

```
[16]: memmap([[0, 0, 0, 1, 0]], dtype=int32)
```

This model makes 5-way one-shot classification task.

Preprocessing the Omniglot Data :

Omniglot is a dataset which is composed of 1623 distinct handwritten characters from 50 different alphabets. The Omniglot dataset aims to cultivate algorithms that mimic human learning processes more closely. Each character was independently drawn by 20 individuals through Amazon's Mechanical Turk platform. Each image is paired with stroke data, presenting sequences of [x,y,t] coordinates where time (t) is measured in milliseconds.

Preprocessing data for the First CNN (U-NET like Architecture):

Since first CNN, takes 1 images and propagates it into 5 , I did not need to create new images. Because python is inherently able to do this. Therefore, I turned 19.280 images into an array with size of 19280 x 1 x 105 x 105 (instances, batch, height, width). Moreover, I created 48.762 strokes png images by utilizing txt data. Then, I put in order each character's strokes from most valuable ones to less valuable by counting the number of ones in the images.

For example : This is the first character's first image from 'Early_Aramaic' alphabet. This is an original image from Omniglot dataset.



When it is plotted with stroke data :



As you can see, colors are different. I plotted with different colors to show the order of strokes. Strokes' order determines which color to use : for example, first stroke is red, second stroke is green, third stroke is blue, etc. This order can be used to capture "*causality*". However, since this project processes data in a different way, I hoped to capture causality in a different way, mainly by taking advantage of the order of strokes in the ground truths that I created as arrays with 5 dimensions.

These components (strokes) were created so as to capture the *compositionality* and *causality* by using spatial information in txt files. For example, I created components below by utilizing txt files.

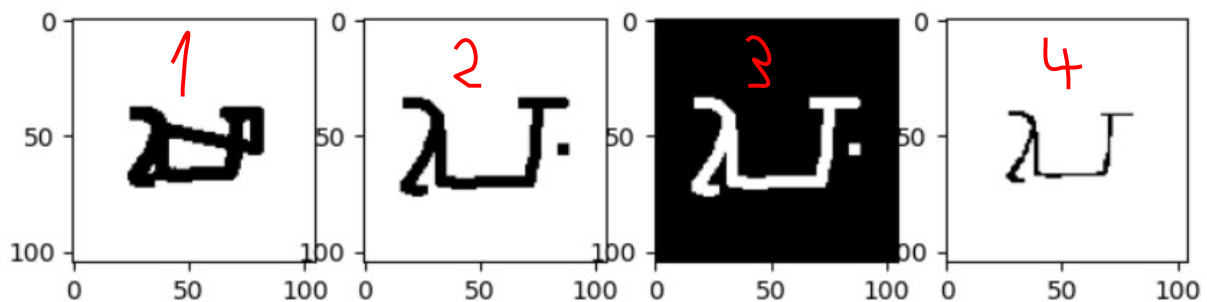


As you can see, they are black. Because this first CNN does not require RGB channels. It needs only one channel. Because our aim is to parse gray-scale images into strokes and then implement one-shot classification, even if we can create images with RGB channels. And these images were created as 105 by 105 pixels, a size which is same as input size. This is a deliberate choice and it was done for two reasons. First, creating pictures with a different size may change the scale of the location of the strokes with respect to the whole area. I wanted to be sure whole pictures overlap on each other to save the scale. However, when I plot them onto each other, I saw that the scale was different. Therefore, I created a new feature matrix different than the original images. I employed the strokes data to create them. In the end, I had created 4 different datasets for pre-train the first CNN. These four different datasets and relevant sizes :

 5_strokes_with_order_NEW.npy	2.1 GB
 new_X_feature_from_ground_truths.npy	1.7 GB
 X_feature_matrix_1_channel_Mostly_Ones.npy	1.7 GB
 X_feature_matrix_1_channel_Mostly_Zeros.npy	1.7 GB
 X_feature_matrix_bold_NEW.npy	1.7 GB

In the end, I had 4 x 19280 feature matrix in total. You can see the differences between them below:

```
plot_img(x_new, 1, 4, 8, 6)
```



Second reason why I created strokes with the size of 105 x 105 is that this approach may facilitate the creating the model's architecture which was going to be planned to be a U-Net like network. Since the skip connections require same size to concatenate different layers, I planned to create strokes images with this shape.

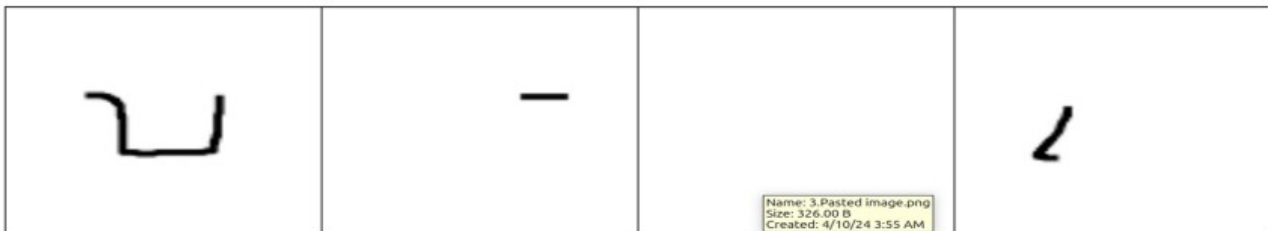
Since each characters of 30 alphabet (train set) have 20 different images, which all are in different folders in the original dataset; after I created strokes with respected names, I put them in one folder and put them in alphabetic order and also created a dictionary of image name versus strokes' names.

```
[19]: {'Alphabet_of_the_Magi_character01_0709_01': ['_stroke1.png',
'_stroke2.png',
'_stroke3.png',
'_stroke4.png'],
'Alphabet_of_the_Magi_character01_0709_02': ['_stroke1.png', '_stroke2.png'],
'Alphabet_of_the_Magi_character01_0709_03': ['_stroke1.png',
'_stroke2.png',
'_stroke3.png']}
```

After I turned each stroke into a png image of 105 by 105 pixels, each stroke image had one channel, their values also were turned into binary values of 0 and 1. I adjusted ones (1s) so as to be the pixels of the characters and zeros (0s) to be the pixels of the empty space. Because I wanted to sum 1s to have an idea of how much information exist in each stroke. For example, the first image for the character of Alphabet_of_the_Magi (character01, first image named as 0709_01) :



When I created stroke images by employing txt data, and since this image's txt data has 4 different stroke data:



Note : Third stroke is empty, therefore image has no value of ones. However, this order is very important, because to capture the *causality* , I needed to stick to the order. I summed all the 1s , to find out which image has the most information to get an order from the most valuable ones to the less. Then, I planned to order them. In the end, I picked 5 of them, because some image's txt data for strokes have more than 10 strokes. Since I wanted my ground truth to be an array of 5 channels. I choose 5 of these strokes, actually the most valuable 5 of them with respect to their information (sum of the 1s), then when I was creating the ground truth images from these 5 strokes, I utilized the original order to capture the *causality*.

For example, an image of the same character (Alphabet_of_the_Magi, character01, 0709_10) have 12 strokes.

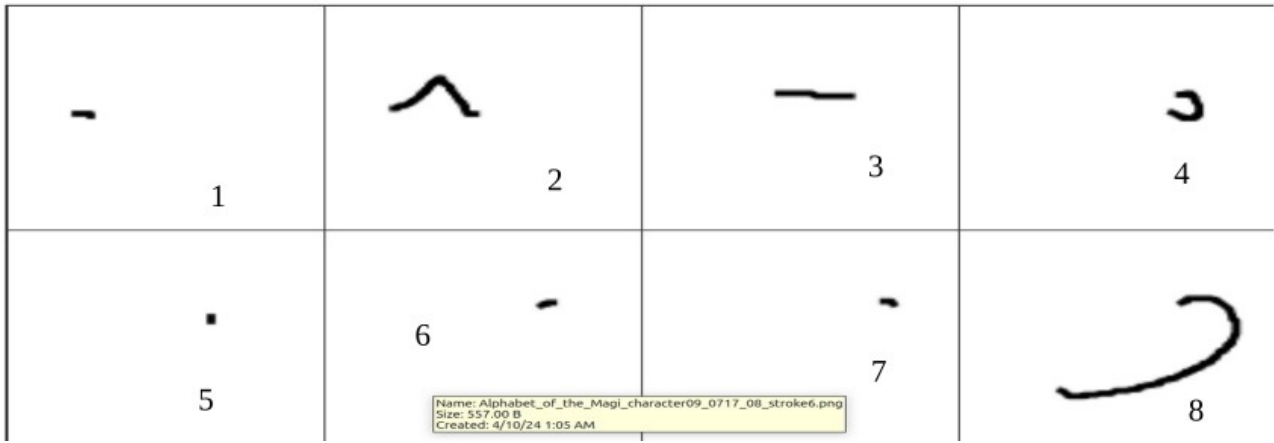
 1	 2	 3			
	 8			 11	

As you can see, some of them do not have any information. Therefore, I put them in order like :

 1	 2	 3	 11	 8	

And I choose the first 5 of the useful ones. In this case, we have only 5 useful strokes and I took all of them. However, sometimes this is not the case and I lose some information, even though it would not be much. After making them in valuability order, I picked them with respect to their original order of : 1, 2, 3, 8, 11

This example below can be understood better :



When I put them in valuability order with respect to their 1s-sum, order was like : 8, 2 , 4 , 3, 6, 7, 1, 5. And I took the first 5 of them : 8, 2, 4, 3 , and 6. Then, I created a ground truth array of 5 channel with original order : 2, 3, 4, 8. Some of the original images (input data) have less than 5 strokes (in txt data) and therefore, some of the channels of their respective ground truth images are empty.

Architecture of the First CNN :

First model is like a U-NET. It takes inputs with 5 dimensions and gives outputs with 5 dimensions. It has a decoder and an encoder part. It has 32 layers in total. I wrote the architecture from scratch. Encoder part starts with 2d convolution layer, and batch normalization layer before the RELU activation function. However, these 3 layers are counted as one layer. Then a similar layer follows. After these two layers, there is a third layer which includes max-pool and drop-out layers. And these layers go until the fifth dropout layer, and then decoder part starts. Decoder part starts with an up-convolution layer and increase the number of pixels at each layer until it reaches to 105 x 105. At all layers, I employed batch normalization layers before RELUs. I also tried Leaky RELU, but it did not made any changes. I utilized 4 concatenate layer for the skip connections. Because I thought that lower-level features are more important than higher-level ones fot the aim for this project. Because the model requires only to parse strokes. The model is complex enough to capture higher-level features. However, this can also lead to lose the lower level features. Therefore, I added skip connections. That was the main reason why I built an architecture like U-NET. I calculated pixels size after each layer in encoder part so as to be in line with pixel size in decoder part. Because any miscalculation gives an error at concatenation layers. All correspondence layers have to have same size of pixels and same size of activation layer dimension. This model has 17.625.477 parameters.

Loss function of the First CNN :

I choose Mean Squared Error loss function. Because I needed to calculate pixel-wise differences between those 5 channels. I wrote many custom loss functions too. However, I could not improve the model further.

Optimizer of the First CNN :

I stick to the Adam optimizer in both models. It works well.

Scheduler of the First CNN :

I employed Step Learning Rate scheduler at first. But then I realized that I can not fully control the training because sometimes I was getting “nan” (not a number) values. And I had to restart kernel to start train again and again. Therefore, I wrote a custom function to train the model both without getting stuck to the nan values and with a decrease in learning rate if needed.

Computational efficiency tools for the First CNN :

I utilized Gradient Scaler and Autocast algorithms to benefit from computational efficiency. Since my laptop’s GPU has only 4 GB memory size, I needed computational tricks in order to process a bigger batch size.

Preprocessing data for the Second CNN (yields vectors as outputs) :

As I mentioned before, I had created a dictionary of image name versus strokes. I already had known that each character of alphabet have 20 different images. When I created data arrays, I stick to this order. For example, first 20 image are belong to the same character and second 20 image are belong to the same character, and this order was saved in creating the arrays.

By knowing this, I wrote an algorithm which can create random numbers like that :

```
[5]: index_data_X[:5]
[5]: memmap([[ 5, 2970, 2600,  2, 4666, 12891],
 [24, 4039, 7114, 11057, 26, 4107],
 [51, 18864, 13270, 14906, 8659, 55],
 [64, 18997, 12092, 15292, 13365, 69],
 [96, 99, 6850, 14243, 12057, 18910]], dtype=int32)
```

As you can see, first number (5) is between 0 and 20, therefore it belongs to the first character, and other 5 columns have only one value between this range which is not 5. That means that if I took an image of a character, I need a similar one to make classification task. However, if this image would be exactly the same, the classification task will be easy. On the other hand, if there would be more than one similar image on the other side, the model will not learn. It will get confused between two choices. Therefore, each value of the first column goes between ranges 0-20, 20-40, 40-60, and other 5 columns have randomly chosen values for classification images , only one of which is the

target. While creating this data, I also created another data to save the indexes to later use. For example, when you look at first row,

```
[5]: memmap([[ 5, 2970, 2600, 2, 4666, 12891],
```

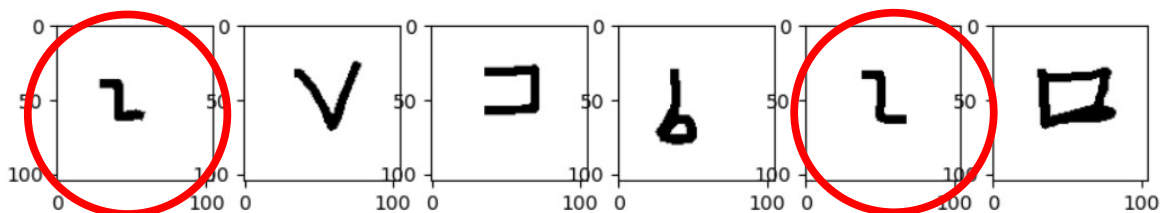
You can see the target image has the index of 2. We count indexes from zero, and we do not include the first image with the value of 5. In the end, we will have a data like :

```
[6]: index_data_y[:5]
```

```
[6]: memmap([2],
             [3],
             [4],
             [4],
             [0]), dtype=int32)
```

However, I turned these values into one-hot vectors of length 5. In the end, I get a dataset like :

```
[15]: plot_img(X[114],1,6, 10,8 )
```



```
[16]: y[114][0]
```

```
[16]: memmap([[0, 0, 0, 1, 0]], dtype=int32)
```

However, I created 3 different feature arrays and 3 different ground truth arrays by employing the algorithm mentioned above. In the end, I had these data sets below, each of which has 100.000 instances. In total, I created 300.000 instances (rows).

 X_new_1.npy	13.2 GB
 X_new_2.npy	13.2 GB
 X_new_3.npy	13.2 GB
 y_1.npy	2.0 MB
 y_2.npy	2.0 MB
 y_3.npy	2.0 MB

Architecture of the Second CNN :

First part of the second model is like the first part of the first model. However, it has 6 layers. At second part, it has 4 fully connected layer. Each layer has many sub-layers. I employed dropout layers in this model too. This model takes inputs with 6 dimension and yields vectors of length of 5. After the last layer, I added a softmax layer. Since ground truths and expected outputs are vectors, I wanted to have a probability distribution of the vectors. This model has 11 million parameters. Even though I increased the number of parameters to nearly 80 million, I could not see any improvement. However, first part of the model had at least 15 layers like U-NET. I thought that this may be a bad idea because I believed that my model needs lower-level features more than higher-level features. Therefore, I removed some of them. And I get only 8 of them in the end. And this decreased the validation loss. I realized that even a cleverly designed architecture with 10 million parameters can work better than a wrongly designed architecture with 80 million parameters.

Loss function of the Second CNN :

I choose Binary Cross Entropy with Logits (BCEWithLogits) loss function. Because I needed to calculate the probability distribution of the vectors. There are some other loss functions like Cross Entropy loss. However, since I am using the autocast and gradient scaler for computational efficiency, they are not compatible with this loss function.

Optimizer choice for the Second CNN :

I stick to the Adam optimizer in both models. It works well.

Scheduler of the Second CNN :

As I mentioned before, because of the nan (not a number) values, I wrote a function to train the model both without getting stuck to the nan values and with a decrease in learning rate if needed.

Computational efficiency tools for the Second CNN :

As I mentioned before, I utilized Gradient Scaler and Autocast algorithms to benefit from computational efficiency.

Hyper-parameter Tuning for Both Models:

Drop-out rate : I used 0.5 rate at first. Then, I decreased it step by step to 0.1. I could not realized much difference between them. This is probably because of that I was constantly changing the architecture. Therefore, complexity of the model was also changing. Since drop-out rate is for increasing generality of the model and preventing the over-fitting. I never encounter any over-fitting situation. Because training loss was behaving like validation loss, every time I train the model.

Learning rate : Actually, I start with 0.01 . Sometimes, it gave nan values, even if I decreased it to very small values like 0.00001. I decreased it, then trained the model again and again. However, I was too tired of being dealt with those parameters. I wrote a function which deals with them instead of me. If the training function gets a nan value, it would start again the loop with a new learning rate of 0.9 times the current learning rate.

Batch Size : I utilized many different batch sizes from 8 to 124. I am not sure why CUDA gave *out-of-memory* error sometimes but not another times.

Optimization :

I changed loss functions many times, even wrote many custom loss functions. However, in the end, I chose the aforementioned loss functions. I created at least 10 different networks and employed many pretrained networks like resnet. However, pretrained networks generally gets inputs with 3 or 4 dimensions. Therefore, I could not use them. I managed to put these two CNNs at work at the end. They are working. However, first CNN for parsing the images into strokes could not work well. On the other hand, second CNN for one-shot classification task works better.

Generalization :

you are expected to apply suitable preprocessing techniques, compare different machine learning techniques, do hyperparameter tuning and report the results.

Your analysis should take into account the three main concerns of machine learning 1) representation 2) optimization 3) generalization,